



**COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS**

ASSIGNMENT PART 1

SEMESTER 2 2025/2026

SUBJECT CODE : CSEB5223
SUBJECT : SOFTWARE CONSTRUCTION & METHODS
DUE DATE : 08 March 2026
NUMBER OF GROUP : GROUP 3
NAME OF MEMBERS :

No.	Full Name	Student ID
1.	AHMAD ILYAS BIN AHMAD IRFAN	SW01083514
2.	WAN NUR SABRINA BINTI BOKHARI	BSW01085928
3.	NUR HIDAYAH BINTI SARWANI	BSW01085937

Table of Contents

1.0 INTRODUCTION	4
1.1 PROJECT BACKGROUND	4
1.2 PROJECT OBJECTIVE	5
2.0 USE CASE AND CLASS DESIGN OF SLMS	7
2.1 USE CASE DIAGRAM.....	7
2.1.1 USE CASE DIAGRAM DESCRIPTION.....	7
2.1.2 ACTORS DESCRIPTION	8
2.1.2.1 STUDENT	8
2.1.2.1 LECTURER.....	8
2.1.2.3 ADMIN	9
2.1.3 SYSTEM BOUNDARY	9
2.1.4 MAIN INTERACTIONS IN THE SYSTEM.....	9
2.1.4.1 STUDENTS INTERACTION	9
2.1.4.2 LECTURERS INTERACTION.....	10
2.1.4.3 ADMIN INTERACTION	10
2.2 CLASS DIAGRAM.....	11
.....	11
3.0 DOCUMENTATION FORMATTING	13
4.0 CODING FILE FORMATTING	14
5.0 NAMING CONVENTIONS.....	15
6.0 FUNCTION AND METHODS MODULARITY	16
7.0 CONTINUOUS INTEGRATION RULES.....	17
8.0 CLASS DESIGN	18
8.1 CLASS DECLARATION.....	18
8.2 ATTRIBUTES	18
.....	18
8.3 CONSTRUCTOR	18
8.4 GETTERS AND SETTERS	19
.....	19
8.5 DISPLAY METHOD.....	19
9.0 COURSE INPUT IMPLEMENTATION	20
9.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES	20
9.2 ‘SUBMIT’ BUTTON TO ADD COURSE PROFILE.....	21
9.3 STORE MULTIPLE COURSES IN ARRAY/LIST	22
9.5 PREVENT ARRAY OUT-OF-BOUND ERRORS.....	23

10.0 SEARCH FUNCTION DEVELOPMENT	24
10.1 IMPLEMENT A LINEAR SEARCH FUNCTION.....	24
10.2 INPUT SEARCH CRITERIA.....	24
10.3 DISPLAY SEARCH RESULT FOR SUCCESFUL SEARCH	25
10.4 DISPLAY FOR UNSUCCESSFUL SEARCH	25
11.0 EDIT FUNCTION DEVELOPMENT.....	27
11.1 IMPLEMENT SEARCH FUNCTION TO FINE THE TARGET COURSE..	27
11.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT ..	28
COURSE CODE).....	28
11.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH.....	28
11.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES	29
12.0 DELETE FUNCTION DEVELOPMENT.....	30
12.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE	30
12.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE	30
12.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH.....	31
12.4 DISPLAY ALL COURSE TO VALIDATE DELETION.....	32
13.0 DATA DISPLAY FUNCTION DEVELOPMENT.....	33
13.1 IMPLEMENT A ‘VIEW ALL’ FUNCTION TO DISPLAY ALL COURSES	33
13.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED	34
FORMAT.....	34
13.3 DISPLAY DATA AFTER SEARCH, EDIT, OR DELETE OPERATIONS .	35
14.0 ANALYSIS AND CHALLENGES	36
14.1 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES	36
14.2 CHALLENGES FACED DURING DEVELOPMENT	37
15.0 EVALUATION CRITERIA	38
15.1 FUNCTIONALITY	38
15.2 DOCUMENTATION / CODING STANDARDS.....	38
15.3 ERROR HANDLING	39
16.0 REFERENCES	40

1.0 INTRODUCTION

1.1 PROJECT BACKGROUND

With the rapid growth of digital technology in education, learning institutions are increasingly relying on online platforms to manage teaching and learning activities. Traditional classroom-based management methods such as manual assignment submission, physical attendance tracking, and paper-based grading are inefficient and difficult to scale.

A Student Learning Management System (SLMS) is a web-based application designed to support digital learning processes. Similar to established platforms like Moodle, an SLMS enables students, lecturers, and administrators to interact within a centralized system. It allows course management, assignment submission, grading, material distribution, and discussion forums to be handled electronically.

The proposed SLMS will serve as a simplified academic platform that manages users, courses, learning materials, assignments, submissions, and discussions. The system emphasizes proper software construction practices, clean architecture, and structured development to ensure maintainability and scalability.

This project is developed as part of the Software Construction & Maintenance course to apply theoretical knowledge into practical system design and implementation.

1.2 PROJECT OBJECTIVE

The main objective of this project is to design and develop a Student Learning Management System (SLMS) by applying proper software construction principles and structured programming practices. A Student Learning Management System is a software platform used in educational institutions to manage academic information such as courses, students, and their relationships. The system helps organize course information, maintain student records, and support academic management activities in a structured and efficient way.

In this project, the SLMS focuses on managing course profiles and student profiles, allowing users to perform operations such as creating, searching, editing, and deleting records. The system also manages the relationship between students and courses, enabling the tracking of course enrolments. The development process follows modular design, object-oriented programming principles, and proper coding standards to ensure the system is scalable, readable, and maintainable.

The specific objectives of this project are:

1. To design the overall structure of the Student Learning Management System using use case diagrams and class diagrams.
2. To develop modules that allow manual input and management of course profiles, including course name, course code, credit hour, course summary, and MS Teams link.
3. To implement a student profile management module that stores and manages student information such as student ID, name, email, and phone number.
4. To implement system functionalities such as searching, editing, deleting, and displaying records for both courses and students.
5. To manage the relationship between students and courses, allowing students to be assigned to courses and courses to maintain a list of enrolled students.
6. To apply software construction best practices, including modular programming, proper naming conventions, documentation standards, and coding standards.
7. To implement error handling and input validation to ensure the system handles invalid inputs and potential errors effectively.

8. To demonstrate the use of version control and continuous integration through Github collaboration during the development process.

Through this project, students will demonstrate their understanding of structured software planning, object-oriented design, modular programming, and disciplined software development practices in building a simplified academic management system similar to existing learning platforms such as Moodle.

2.0 USE CASE AND CLASS DESIGN OF SLMS

2.1 USE CASE DIAGRAM

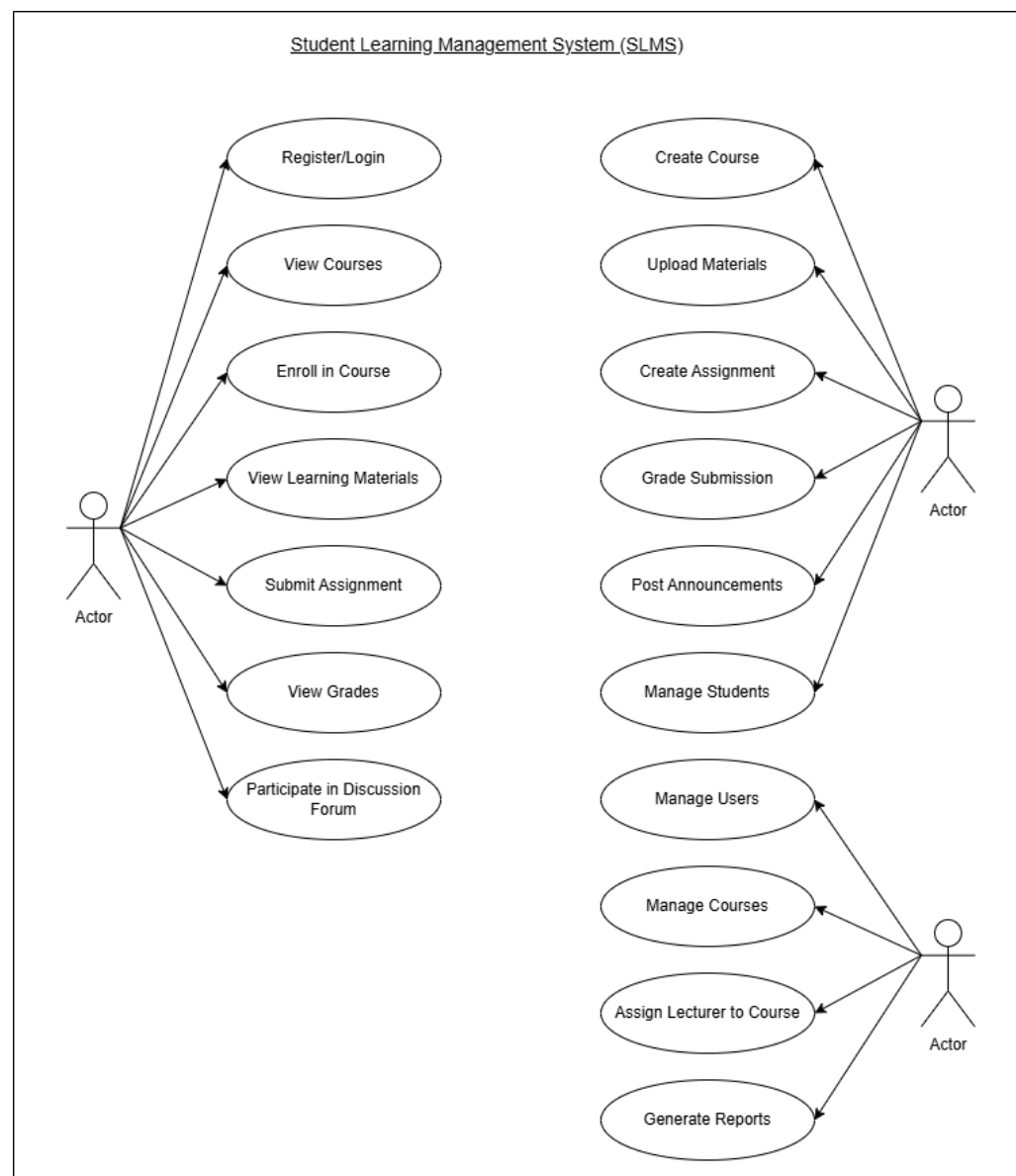
2.1.1 USE CASE DIAGRAM DESCRIPTION

A use case describes how users interact with a system to achieve a specific goal. It represents a sequence of actions between an actor and the system that leads to a useful outcome.

Use cases help developers understand:

- System functionality
- User requirements
- System interactions
- Scope of the system

Use case diagrams provide a high-level overview of the system's functionality and how external actors interact with it.



2.1.2 ACTORS DESCRIPTION

2.1.2.1 STUDENT

A Student is a user who participates in courses and learning activities. Students can access course materials, submit assignments, participate in discussions, and view grades.

Student responsibilities include:

- Register and log into the system
- Enrol in available courses
- View course materials
- Submit assignments
- Participate in discussion forums
- View grades and feedback

2.1.2.1 LECTURER

A Lecturer is responsible for managing courses and delivering learning content to students.

Lecturer responsibilities include:

- Create and manage courses
- Upload learning materials
- Create assignments
- Grade student submissions
- Provide feedback
- Moderate discussion forums

2.1.2.3 ADMIN

An Admin is responsible for managing the system and controlling overall operations.

Admin responsibilities include:

- Manage user accounts
- Manage courses
- Assign lecturers to courses
- Monitor system usage
- Generate system reports

2.1.3 SYSTEM BOUNDARY

The system boundary represents the limits of the SLMS system. It separates the internal system functionalities from external actors.

In the use case diagram:

- All use cases are located inside the SLMS system boundary.
- Actors such as Student, Lecturer, and Admin are outside the boundary because they interact with the system.

This helps define:

- What the system is responsible for
- What actions are performed by users outside the system

2.1.4 MAIN INTERACTIONS IN THE SYSTEM

The SLMS system supports several key interactions between actors and the system.

2.1.4.1 STUDENTS INTERACTION

Students interact with the system to manage their learning activities.

Main interactions include:

- Registering and logging into the system
- Enrolling in courses
- Viewing course materials
- Submitting assignments
- Participating in discussions

- Viewing grades

2.1.4.2 LECTURERS INTERACTION

Lecturers interact with the system to manage course content and evaluate students.

Main interactions include:

- Creating courses
- Uploading learning materials
- Creating assignments
- Grading student submissions
- Providing feedback
- Managing discussion forums

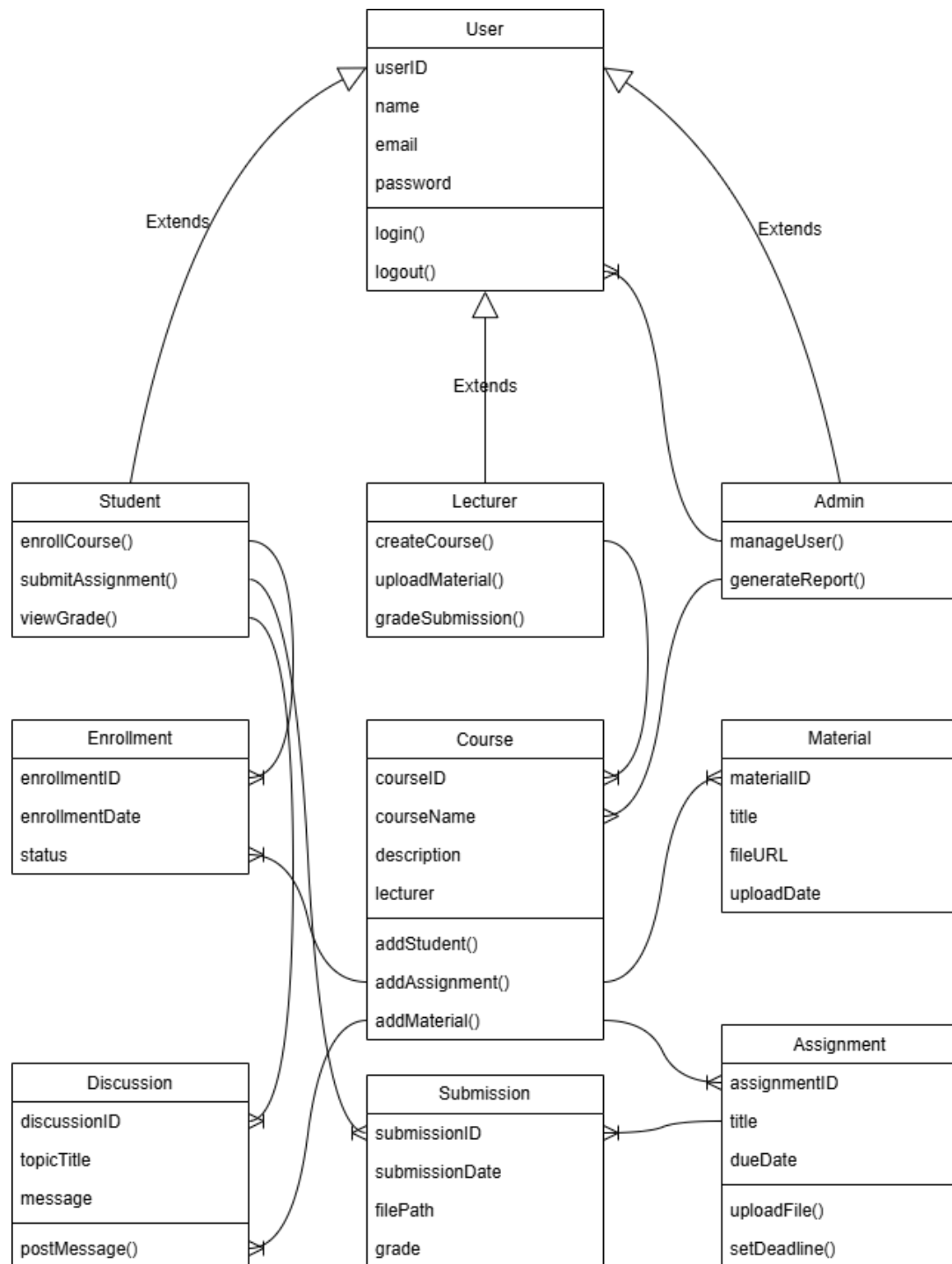
2.1.4.3 ADMIN INTERACTION

Admins interact with the system to maintain and control the platform.

Main interactions include:

- Managing users (students and lecturers)
- Managing courses
- Assigning lecturers to courses
- Generating system reports

2.2 CLASS DIAGRAM



Object-Oriented Approach

- Used inheritance to promote reusability and reduce redundancy.

Clear Role Separation

Student, Lecturer, and Admin responsibilities are separated to enforce:

- Security
- Maintainability
- Scalability

Proper Relationship Modelling

- Association used for independent entities.
- Aggregation used when object belongs to another but can exist independently.
- Composition used when lifecycle dependency exists.

Extensibility

- Future roles (e.g., TeachingAssistant) can extend User easily.

Maintainability

- Modular class structure makes:
- Debugging easier
- Testing easier
- Feature addition safer

3.0 DOCUMENTATION FORMATTING

The SLMS documentation will follow the following format:

- Font: Times New Roman
- Font Size: 12
- Line spacing: 1.5
- Heading format:
 - 1.0 Main Heading (Bold)
 - 1.1 Subheading
- Alignment: Justified
- UML diagrams labelled properly
- Each section must include:
 - Purpose
 - Description
 - Example (if needed)

For code documentation, use:

- File header comment
- Function header comment
- Inline comments for complex logic

Example:

```
"""
File: course.py
Author: Team SLMS
Description: Handles course-related operations.
Date: 21 Feb 2026
"""
```

4.0 CODING FILE FORMATTING

- Indentation uses 4 spaces.
- Opening and closing brackets must align.
- Comments are required before functions.
- Files organised into folders.

Example:

```
if (loginSuccess) {  
    openDashboard();  
}
```

5.0 NAMING CONVENTIONS

Element	Convention	Example
Class	PascalCase	StudentAccount
Function	camelCase	submitAssignment()
Variable	snake_case	student_name
Constant	UPPER_CASE	MAX_LOGIN_ATTEMPT
File Name	lowercase	course.py

Term	Meaning
SLMS	Student Learning Management System
LMS	Learning Management System
Enrollment	Student registered in course
Submission	Assignment uploaded by student

6.0 FUNCTION AND METHODS MODULARITY

The system is designed using a modular approach where each function and method performs a single, well-defined task. Large functionalities are broken down into smaller and reusable functions to improve readability, maintainability, and scalability of the system.

For example, user authentication is separated into different functions such as input validation, credential verification, and session handling. This modular design allows easier debugging, testing, and future enhancement of the system.

Each module is designed to minimize dependency on other modules, allowing changes to be made with minimal impact on the overall system. This approach supports team collaboration, as different members can work on different modules independently.

7.0 CONTINUOUS INTEGRATION RULES

The following continuous integration rules are implemented to ensure effective collaboration and maintain code quality throughout the development of the Student Learning Management System (SLMS):

1. Each team member must develop features in a separate feature branch.
2. Direct commits to the main branch are prohibited.
3. All changes must be reviewed and approved by the team leader before merging.
4. Clear and meaningful commit messages are required for every commit.
5. Code must be tested to ensure it functions correctly before integration.
6. GitHub is used as the main platform for version control and collaboration.

These rules help reduce integration issues and improve the overall quality of the system.

Example:

Added assignment submission module.

Fixed login validation error.

8.0 CLASS DESIGN

8.1 CLASS DECLARATION

```
3 public class Course {
```

Explanation:

- Course is the blueprint for a course.
- All course-related data and functions are organized inside this class.
- Every new course you add will be an object of this class.

8.2 ATTRIBUTES

```
4 // Attributes
5 private String courseName;
6 private String courseCode;
7 private int creditHour;
8 private String summary;
9 private String msTeamsLink;
```

Explanation:

- These are the data for each course.
- private means they are hidden from outside, to protect data integrity.
- courseCode cannot be changed after creation - this ensures uniqueness.

8.3 CONSTRUCTOR

```
11 // Constructor
12 public Course(String courseName, String courseCode, int creditHour, String summary, String msTeamsLink) {
13     this.courseName = courseName;
14     this.courseCode = courseCode;
15     this.creditHour = creditHour;
16     this.summary = summary;
17     this.msTeamsLink = msTeamsLink;
18 }
```

Explanation:

- Constructor is used to create a new course object with all required data.

8.4 GETTERS AND SETTERS

```
20 // Getters and Setters
21 public String getCourseName() { return courseName; }
22 public void setCourseName(String courseName) { this.courseName = courseName; }
23
24 public String getCourseCode() { return courseCode; }
25
26 public int getCreditHour() { return creditHour; }
27 public void setCreditHour(int creditHour) { this.creditHour = creditHour; }
28
29 public String getSummary() { return summary; }
30 public void setSummary(String summary) { this.summary = summary; }
31
32 public String getMsTeamsLink() { return msTeamsLink; }
33 public void setMsTeamsLink(String msTeamsLink) { this.msTeamsLink = msTeamsLink; }
```

Explanation:

- Getters → read/access the data.
- Setters → update/change the data (except courseCode).
- This keeps the data safe and controlled, a principle called encapsulation.

8.5 DISPLAY METHOD

```
35 // Display all course attributes
36 public void displayCourse() {
37     System.out.println("Course Name      : " + courseName);
38     System.out.println("Course Code     : " + courseCode);
39     System.out.println("Credit Hour  : " + creditHour);
40     System.out.println("Summary       : " + summary);
41     System.out.println("MS Teams Link : " + msTeamsLink);
42     System.out.println("-----");
43 }
44 }
```

Explanation:

- Method to show all course info in a clear format.
- Useful for View All, Search, Edit, and Delete functions.

9.0 COURSE INPUT IMPLEMENTATION

9.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES

```
57 private void addCourse() {
58     System.out.print("Enter Course Name: ");
59     String name = sc.nextLine();

61     String code;
62     while (true) {
63         System.out.print("Enter Course Code: ");
64         code = sc.nextLine();
65         if (findCourseIndex(code) != -1) {
66             System.out.println("Course code already exists! Enter a unique code.");
67         } else break;
68     }

70     int credit;
71     while (true) {
72         System.out.print("Enter Credit Hour: ");
73         if (sc.hasNextInt()) {
74             credit = sc.nextInt();
75             sc.nextLine();
76             if (credit > 0) break;
77             else System.out.println("Credit hour must be positive!");
78         } else {
79             System.out.println("Invalid input! Enter a number.");
80             sc.next();
81         }
82     }

84     System.out.print("Enter Course Summary: ");
85     String summary = sc.nextLine();
86
87     System.out.print("Enter MS Teams Link: ");
88     String link = sc.nextLine();

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 1
Enter Course Name: SOFTWARE SECURITY
Enter Course Code: CSEB5113
Enter Credit Hour: 3
Enter Course Summary: SECURITY
Enter MS Teams Link: https://teams.microsoft.co
```

Explanation:

- These are the fields where the user enters course details.
- User inputs: course name, course code, credit hours, summary, and MS Teams link.

9.2 'SUBMIT' BUTTON TO ADD COURSE PROFILE

```
90 |         courses.add(new Course(name, code, credit, summary, link));  
91 |         System.out.println("Course added successfully!");  
92 |     }
```

```
Enter Course Name: SOFTWARE SECURITY  
Enter Course Code: CSEB5113  
Enter Credit Hour: 3  
Enter Course Summary: SECURITY  
Enter MS Teams Link: https://teams.mi  
Course added successfully!
```

Explanation:

- This is equivalent to a “Submit” button.
- It creates a new Course object and adds it to the ArrayList.
- The confirmation message shows that the course has been successfully added.

9.3 STORE MULTIPLE COURSES IN ARRAY/LIST

```
8      // Store multiple courses
      private ArrayList<Course> courses = new ArrayList<>();
      private Scanner sc = new Scanner(System.in);

11  /**
    --- COURSE PROFILE SYSTEM ---
    1. Add Course
    2. Search Course
    3. Edit Course
    4. Delete Course
    5. View All Courses
    0. Exit
    Choose an option: 5

    --- All Courses ---
    Course Name      : SOFTWARE SECURITY
    Course Code      : CSEB5113
    Credit Hour      : 3
    Summary          : SECURITY
    MS Teams Link    : https://teams.microsoft.com/l/t
    -----
    Course Name      : SOFTWARE CONSTRUCTION & METHOD
    Course Code      : CSEB5223
    Credit Hour      : 3
    Summary          : GITHUB AND CODING
    MS Teams Link    : https://teams.microsoft.com/l/t
    -----
```

Explanation:

- ArrayList stores all courses added by the user.
- Can store any number of courses dynamically, no fixed limit.

9.5 PREVENT ARRAY OUT-OF-BOUND ERRORS

```
101 |                                     int index = findCourseIndex(code);
102 |
103 | ☐ if (index != -1) {
104 |     System.out.println("Course found:");
105 |     courses.get(index).displayCourse();
106 | ☐ } else {
107 |     System.out.println("Course not found.");
108 | }
109 | }
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 2
Enter Course Code to search: SCM
Course not found.
```

Explanation:

- Before accessing a course from the list, the program checks if the course exists.
- This prevents array out-of-bound errors (trying to access a non-existing index).

10.0 SEARCH FUNCTION DEVELOPMENT

10.1 IMPLEMENT A LINEAR SEARCH FUNCTION

```
205 private int findCourseIndex(String code) {  
206     for (int i = 0; i < courses.size(); i++) {  
207         if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {  
208             return i;  
209         }  
210     }  
211     return -1;  
212 }  
213 }
```

Explanation:

- This method implements a linear search algorithm.
- The program loops through the ArrayList of courses one by one.
- It compares the course code entered by the user with each course stored in the system.
- If a matching course code is found, the index of that course is returned.
- If no match is found, the method returns -1.

10.2 INPUT SEARCH CRITERIA

```
99 System.out.print("Enter Course Code to search: ");  
100 String code = sc.nextLine();  
101 int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: CSEB5223  
Invalid input! Enter a number: 2  
Enter Course Code to search: CSEB5223
```

Explanation:

- The user enters a course code as the search criterion.
- The program then calls the linear search method (findCourseIndex) to locate the course in the list.

10.3 DISPLAY SEARCH RESULT FOR SUCCESSFUL SEARCH

```
103  if (index != -1) {  
104      System.out.println("Course found:");  
105      courses.get(index).displayCourse();  
    }
```

```
Enter Course Code to search: CSEB5223  
Course found:  
Course Name      : SCM  
Course Code      : CSEB5223  
Credit Hour      : 3  
Summary          : GITHUB  
MS Teams Link    : https://teams.microsoft.com/1/1  
-----
```

Explanation:

- If the course exists in the system, the program retrieves the course object using its index.
- The method `displayCourse()` is called to show all course attributes, including:
 - Course Name
 - Course Code
 - Credit Hour
 - Course Summary
 - MS Teams Link

10.4 DISPLAY FOR UNSUCCESSFUL SEARCH

```
106  } else {  
107      System.out.println("Course not found.");  
108  }  
109  }
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 2  
Enter Course Code to search: CSEB5113  
Course not found.
```

Explanation:

- If the entered course code does not exist, the program shows a message “Course not found.”
- This ensures the system handles invalid searches properly without crashing.

11.0 EDIT FUNCTION DEVELOPMENT

11.1 IMPLEMENT SEARCH FUNCTION TO FINE THE TARGET COURSE

```
116 |         System.out.print("Enter Course Code to edit: ");
117 |         String code = sc.nextLine();
118 |         int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEB5223
```

Explanation:

- The system first asks the user to enter the course code of the course that needs to be edited.
- The program then calls the linear search method `findCourseIndex()` to locate the course in the `ArrayList`.
- If the course exists, its index will be returned for further editing.

11.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT COURSE CODE)

```
121 Course c = courses.get(index);
122 System.out.println("Editing Course: " + c.getCourseCode());
123
124 System.out.print("New Course Name: ");
125 c.setCourseName(sc.nextLine());
126
127 int credit;
128 while (true) {
129     System.out.print("New Credit Hour: ");
130     if (sc.hasNextInt()) {
131         credit = sc.nextInt();
132         sc.nextLine();
133         if (credit > 0) break;
134         else System.out.println("Credit hour must be positive!");
135     } else {
136         System.out.println("Invalid input! Enter a number.");
137         sc.next();
138     }
139 }
140 c.setCreditHour(credit);
141
142 System.out.print("New Summary: ");
143 c.setSummary(sc.nextLine());
144
145 System.out.print("New MS Teams Link: ");
146 c.setMsTeamsLink(sc.nextLine());
```

```
Editing Course: CSEB5223
New Course Name: SOFTWARE CONSTRUCTION & METHOD
New Credit Hour: 3
New Summary: GITHUB AND CODING
New MS Teams Link: https://teams.microsoft.com/1
```

Explanation:

- Once the course is found, the program retrieves the course object from the ArrayList.
- The user is allowed to edit all course attributes, including:
 - Course Name
 - Credit Hour
 - Course Summary
 - MS Teams Link
- However, the course code cannot be edited because it acts as a unique identifier for each course.

11.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```

150 |         } else {
151 |             System.out.println("Course not found.");
152 |         }
153 |     }

```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEB5123
Course not found.

```

Explanation:

- If the course code entered by the user does not exist, the system will display “Course not found.”
- This ensures the program handles invalid input without causing system errors.

11.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES

```

148 |         System.out.println("Course updated successfully!");
149 |         c.displayCourse();

```

```

Course updated successfully!
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code     : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link   : https://teams.microsoft.com/l/team/19'
-----

```

Explanation:

- After editing the course information, the system displays the updated course details.
- This allows the user to verify that the changes were successfully applied.

12.0 DELETE FUNCTION DEVELOPMENT

12.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE

```
160 System.out.print("Enter Course Code to delete: ");
161 String code = sc.nextLine();
162 int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 4
Enter Course Code to delete: CSEB3123
```

Explanation:

- The system asks the user to enter the course code of the course that needs to be deleted.
- The program uses the findCourseIndex() method (linear search) to locate the course in the ArrayList.
- If the course exists, the program retrieves its index for further action.

12.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE

```
165 Course c = courses.get(index);
166 System.out.println("Course found:");
167 c.displayCourse();
168
169 System.out.print("Confirm deletion? (Y/N): ");
170 String confirm = sc.nextLine();
```

```
Course found:
Course Name    : SOFTWARE ENGINEERING
Course Code    : CSEB3123
Credit Hour    : 3
Summary        : SDLC
MS Teams Link  : https://teams.microso!
-----
Confirm deletion? (Y/N): Y
```

Explanation:

- If the course is found, the system displays the complete course attributes using the `displayCourse()` method.
- This allows the user to verify the correct course before deleting it.
- The program then asks for confirmation (Y/N) before proceeding with deletion.

12.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```
180 } else {  
181     System.out.println("Course not found.");  
182 }  
183 }
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 4  
Enter Course Code to delete: CSEB5113  
Course not found.
```

Explanation:

- If the entered course code does not exist, the system displays the message “Course not found.”
- This prevents the program from attempting to delete a non-existing course.

12.4 DISPLAY ALL COURSE TO VALIDATE DELETION

```
172 |         courses.remove(index);
173 |         System.out.println("Course deleted successfully!");
174 |
175 |         // Display all courses after deletion
176 |         viewAllCourses();
Course deleted successfully!

--- All Courses ---
Course Name      : SOFTWARE SECURITY
Course Code      : CSEB5113
Credit Hour     : 3
Summary         : SECURITY
MS Teams Link    : https://teams.microsoft.com/l/te
-----
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code      : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link    : https://teams.microsoft.com/l/te
-----
```

Explanation:

- If the user confirms the deletion, the course is removed from the ArrayList using `remove(index)`.
- The program then displays all remaining courses using `viewAllCourses()` to confirm that the selected course has been successfully deleted.

13.0 DATA DISPLAY FUNCTION DEVELOPMENT

13.1 IMPLEMENT A ‘VIEW ALL’ FUNCTION TO DISPLAY ALL COURSES

```
46 | | | | | case 5: viewAllCourses(); break;
```

```
--- COURSE PROFILE SYSTEM ---
```

```
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 5
```

Explanation:

- The system provides a menu option (5 – View All Courses) that acts like a “View All” button.
- When the user selects this option, the program calls the viewAllCourses() method.
- This method displays all courses currently stored in the system.

13.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED FORMAT

```
188  private void viewAllCourses() {
189      if (courses.isEmpty()) {
190          System.out.println("No courses available.");
191      } else {
192          System.out.println("\n--- All Courses ---");
193          for (Course c : courses) {
194              c.displayCourse();
195          }
196      }
197  }
```

--- All Courses ---

Course Name	: SOFTWARE SECURITY
Course Code	: CSEB5113
Credit Hour	: 3
Summary	: SECURITY
MS Teams Link	: https://teams.microsoft.com/l/tea

Course Name	: SOFTWARE CONSTRUCTION & METHOD
Course Code	: CSEB5223
Credit Hour	: 3
Summary	: GITHUB AND CODING
MS Teams Link	: https://teams.microsoft.com/l/tea

Explanation:

- This method displays all courses stored in the ArrayList.
- It first checks whether the list is empty.
- If courses exist, the system loops through the list and calls displayCourse() for each course.
- The displayCourse() method ensures the information is shown in a clear and organized format.

13.3 DISPLAY DATA AFTER SEARCH, EDIT, OR DELETE OPERATIONS

```
172 |         courses.remove(index);
173 |         System.out.println("Course deleted successfully!");
174 |
175 |         // Display all courses after deletion
176 |         viewAllCourses();

Course deleted successfully!

--- All Courses ---
Course Name      : SOFTWARE SECURITY
Course Code      : CSEB5113
Credit Hour     : 3
Summary         : SECURITY
MS Teams Link    : https://teams.microsoft.com/l/te
-----
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code      : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link    : https://teams.microsoft.com/l/te
-----
```

Explanation:

- After certain operations such as delete, the system automatically calls the `viewAllCourses()` method.
- This allows users to see the updated list of courses immediately.
- It helps confirm that the search, edit, or deletion operation was successfully performed.

14.0 ANALYSIS AND CHALLENGES

14.1 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES

The development of the Course Profile System follows several software construction best practices to ensure the system is organized, maintainable, and reliable. First, the program applies object-oriented programming principles by separating the system into two main classes: Course and CourseManager. The Course class is responsible for storing course attributes such as course name, course code, credit hour, course summary, and MS Teams link. The CourseManager class manages the system operations including adding, searching, editing, deleting, and displaying course profiles.

Encapsulation is also applied by declaring class attributes as private and providing getter and setter methods to control access and modification of the data. This helps protect the integrity of the data and ensures that course information can only be modified through proper methods.

Additionally, the program uses modular programming, where each functionality is implemented in separate methods such as addCourse(), searchCourse(), editCourse(), deleteCourse(), and viewAllCourses(). This makes the program easier to understand, maintain, and debug. The use of ArrayList also allows the system to store multiple course objects dynamically without needing a fixed-size array.

14.2 CHALLENGES FACED DURING DEVELOPMENT

During the development of the Course Profile System, several challenges were encountered. One of the main challenges was implementing the linear search function to correctly locate courses using the course code. It required careful handling to ensure that the system returns the correct index or an appropriate message if the course does not exist.

Another challenge was handling user input validation. The program needed to ensure that the user enters valid data, such as numeric values for credit hours. This was handled using methods such as `hasNextInt()` to prevent invalid input from causing runtime errors.

Additionally, ensuring that duplicate course codes are not allowed was another challenge. Since the course code acts as a unique identifier, the program had to check the existing course list before allowing a new course to be added. Implementing proper checks for search, edit, and delete operations was also important to avoid accessing invalid indexes and causing array out-of-bound errors.

15.0 EVALUATION CRITERIA

15.1 FUNCTIONALITY

The Course Profile System successfully supports the required functionalities for managing course profiles. The system allows users to input course information, including course name, course code, credit hour, course summary, and MS Teams link. These course profiles are stored in an ArrayList and can be displayed using the View All function.

The system also provides features to search, edit, and delete course profiles. The search function uses a linear search algorithm to locate courses based on their course code. The edit function allows users to modify course details except for the course code, which remains unchanged as a unique identifier. The delete function allows users to remove a selected course after confirmation. These functions ensure that the system can effectively manage course information.

15.2 DOCUMENTATION / CODING STANDARDS

The program follows appropriate documentation and coding standards to improve readability and maintainability. Comments are included throughout the code to explain the purpose of classes and methods. Meaningful variable names and method names such as `addCourse()`, `searchCourse()`, and `viewAllCourses()` help make the code easier to understand.

The program structure is also well organized by separating responsibilities into different classes. The `Course` class focuses on representing course data, while the `CourseManager` class handles system operations and user interaction. This separation of responsibilities follows good software design principles.

15.3 ERROR HANDLING

The program includes several error handling techniques to ensure that the system runs smoothly even when users provide invalid input. For example, the program checks whether the user enters a valid numeric value when inputting credit hours by using `hasNextInt()`.

The system also checks whether a course code already exists before adding a new course to prevent duplicate entries. In addition, before performing search, edit, or delete operations, the program verifies whether the course exists by using the `findCourseIndex()` method. If the course is not found, the system displays an appropriate message such as “Course not found.” These mechanisms help prevent runtime errors and improve the overall reliability of the system.

16.0 REFERENCES

Moodle Official Website. <https://moodle.org/>

Moodle Codebase Repository. <https://github.com/moodle/moodle>